

Experimental Methods in Computer Science

Experiment Design

André Miguel Correia Cardoso 2022222265

Luis Miguel Ferreira Vaz 2022214707

Pedro Costa 2022220304

University of Coimbra, Mestrado em Engenharia Informática 2025/2026

Contents

1	Introduction	4
2	Experimental Design	4
2.1		4
3	Bug Finding	4
3.1	Experimental Design	5
3.1.1	Research Question	5
3.1.2	Independent Variables	5
3.1.3	Dependent Variables	5
3.1.4	Controlled Variables	5
4	Code Generation	6
4.1	Context and Motivation	6
4.2	Dataset and Problem Selection	6
4.3	Prompting and Generation Procedure	6
4.4	Evaluation Methodology	6
4.5	Performance Metrics	6
4.6	Experimental Controls	7
4.7	Research Questions	7
4.8	Variables	7
4.8.1	Independent Variables	7
4.8.2	Dependent Variables	8
4.8.3	Controlled Variables	8
4.9	Results and Correctness Analysis	9
4.10	Statistical Comparison	10
4.10.1	(i) Hypothesis	10
4.10.2	(ii) Statistical Test Selection	10
4.10.3	(iii) Data Characteristics	10
4.10.4	(iv) Test Computation	11
4.10.5	(v) Results	11

4.10.6 (vi) Decision and Conclusion	11
---	----

1 Introduction

Large Language Models (LLMs) are increasingly being integrated into software development workflow. As their use becomes more widespread, it is important to evaluate the extent to which developers can rely on these tools in practical software engineering contexts.

This report was developed as part of the Experimental Methods in Computer Science curricular unit and addresses the following central problem statement:

Can developers rely on LLM-based tools to support software development activities?

Given the wide scope of this question, the experiment focuses on two specific areas of software development: Code Generation and Code Review (more specifically Bug Finding). These two areas were studied separately, using distinct datasets and statistical analysis methods suited to each task.

Despite these differences, both parts of the experiment share a common setting: they are based on the Python programming language and evaluate the same LLM models. This allows for a structured comparison of model performance across two relevant software development activities, contributing to a broader understanding of the capabilities and limitations of LLM-based tools.

2 Experimental Design

2.1

3 Bug Finding

Code Review, and more specifically Bug Finding, refers to the practice of providing an LLM with source code and asking it to inspect the code for potential defects. Depending on the experimental setup, this process may involve supplying only the relevant code files, or it may include additional contextual information such as failing tests, execution outputs, error messages, or brief descriptions of the expected behaviour.

This experiment focuses on the bug detection capabilities of the selected LLMs. In particular, the models are tasked with analysing Python source code and identifying the locations of real bugs. The objective is to evaluate how effectively each model can support developers in detecting defects during code review activities.

3.1 Experimental Design

3.1.1 Research Question

The research question for this experiment is:

How does the choice of Large Language Model affect bug-finding performance in real-world Python bugs?

3.1.2 Independent Variables

The independent variables are:

- **LLM Model (2 levels):**
- **Language (1 level) :**
 - Python

Even though there is only one language being used in the bug, it is still considered an independent variable.

3.1.3 Dependent Variables

The dependent variables used in the statistical comparison are:

- **Recall:** Fraction of the real bugs identified.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:** F1-score combines recall and precision into a single metric that penalises both missed bugs and incorrectly reported bugs:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

Precision was excluded, since, due to its formula, when an LLM reports zero bugs, it will generate a division by 0. Since F1-score is the harmonic mean between Recall and Precision we decided to opt for that dependent variable.

3.1.4 Controlled Variables

In order to reduce unnecessary variables and isolate each LLM's performance, we kept the following variable constant:

- **Scenarios:** Both LLMs are fed the same bug scenarios

- **Prompt:** All prompts follow the same format. They just vary from scenario to scenario in the Source Files section, since they include that specific scenario’s buggy code.
- **New chat for each submission:** In order to avoid outputs based on previous context, each submission is made in a new chat.

4 Code Generation

4.1 Context and Motivation

This experiment evaluated the reliability of Large Language Models (LLMs) in generating SQL queries. SQL was selected because it provides a standardized environment in which generated solutions can be automatically executed and validated against predefined test cases.

4.2 Dataset and Problem Selection

A dataset consisting of 45 SQL programming problems was collected from the LeetCode platform. The selected problems were evenly distributed across three difficulty levels (Easy, Medium, and Hard), enabling the evaluation of model performance under varying levels of complexity.

4.3 Prompting and Generation Procedure

For each problem, both LLMs received the same prompt template and were instructed to generate a single SQL query as the solution. Each problem was submitted independently, and no interaction history was shared between problems.

Each model was allowed a single attempt per problem. This design choice was made to evaluate first-response reliability and to avoid bias introduced by iterative refinement or multiple attempts.

4.4 Evaluation Methodology

The generated solutions were submitted to the LeetCode evaluation platform, which automatically executed each query against a set of hidden test cases. The platform determined whether each solution was accepted or rejected based on its execution results.

The evaluation was based on the acceptance status rather than direct comparison of query syntax. This ensures that semantically equivalent queries producing identical results are considered correct.

4.5 Performance Metrics

In addition to correctness, execution time reported by the platform was also recorded. This metric is used to analyze the query’s efficiency, enabling a comparative analysis of computational

performance between models.

Although execution time is influenced by factors such as query structure and database optimization strategies, it provides a practical indicator of relative efficiency between generated solutions.

4.6 Experimental Controls

To ensure experimental consistency, all problems were presented using the same prompt structure. Only the problem statement and input/output specifications varied between tasks.

Furthermore, each problem was submitted in an independent conversation context, preventing information from previous interactions from influencing subsequent responses. As a result, every generated solution constitutes an independent observation.

4.7 Research Questions

The experiment was designed to investigate the reliability of Large Language Models (LLMs) in SQL code generation. Specifically, the study aims to address the following research questions:

- **RQ1:** Is there a statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash?
- **RQ2:** Does problem difficulty significantly influence the correctness of SQL queries generated by Large Language Models?

4.8 Variables

The experimental design includes two independent variables and multiple dependent variables used to evaluate the reliability and performance of Large Language Models (LLMs) in SQL code generation.

4.8.1 Independent Variables

IV1: LLM Model

This variable represents the Large Language Model used to generate SQL queries. It has two levels:

- ChatGPT Mini
- Gemini Flash

IV2: Problem Difficulty

This variable represents the difficulty level of the SQL programming problems selected from the dataset. It has three levels:

- Easy
- Medium
- Hard

4.8.2 Dependent Variables

DV1: Query Outcome Classification (Primary Outcome)

This variable classifies the outcome of each generated SQL query based on its execution and correctness. It is defined as a categorical variable with the following levels:

- **Correct:** the query executes successfully and passes all hidden test cases
- **Incorrect Logic:** the query executes successfully but fails one or more hidden test cases due to logical or constraint violations
- **Syntax or Execution Error:** the query fails to execute due to syntax errors or runtime issues

This classification allows a more granular analysis of failure types beyond binary correctness.

DV2: Execution Success

This variable captures whether the generated SQL query is syntactically valid and executable. It is defined as:

- Executes successfully
- Execution error

DV3: Execution Time (Secondary Performance Metric)

This variable records the execution time reported by the evaluation platform for each query. It is treated as a continuous variable and used as a proxy for query efficiency. It is not used as a measure of correctness, but instead as a secondary performance indicator to compare computational efficiency across models.

4.8.3 Controlled Variables

Several factors that could influence the quality of the generated solutions were kept constant throughout the experiment to ensure that observed differences could be attributed to the independent variables under investigation.

CV1: Prompt Template

All problems were presented using the same prompt template. Only the problem statement and the associated input/output specifications varied between tasks.

CV2: Number of Generation Attempts

Each model was allowed a single attempt per problem. No retries, prompt refinements, or iterative interactions were permitted.

CV3: Evaluation Platform

All generated solutions were evaluated using the LeetCode platform under the same evaluation procedure and hidden test cases.

CV4: Conversation Context

Each problem was submitted in an independent conversation context. This ensured that information from previous interactions could not influence subsequent responses.

4.9 Results and Correctness Analysis

The generated SQL solutions were evaluated on 60 problems per model. Since the evaluation platform returns a binary outcome (Accepted or Not Accepted), correctness was measured as the proportion of accepted solutions:

$$\text{Accuracy} = \frac{\text{Accepted Solutions}}{\text{Total Problems}}$$

Table 1 presents the overall correctness achieved by each model.

Model	Accepted	Total	Accuracy
ChatGPT	58	60	96.67%
Gemini	57	60	95.00%

Table 1: Overall correctness of the SQL code generation models.

Both models achieved high correctness rates, with ChatGPT obtaining a marginally higher accuracy than Gemini. The difference corresponds to a single additional accepted solution.

To evaluate the impact of problem complexity, the results were further analyzed according to the predefined difficulty levels.

Difficulty	ChatGPT	Gemini	Total Problems
Easy	20/20 (100.00%)	20/20 (100.00%)	20
Medium	20/20 (100.00%)	19/20 (95.00%)	20
Hard	16/20 (80.00%)	17/20 (89.47%)	20

Table 2: Correctness by difficulty level.

The results indicate a progressive decrease in correctness as problem difficulty increases. Both models solved all Easy problems successfully, while the Hard category produced the lowest

acceptance rates. This trend suggests that problem complexity has a measurable impact on SQL code generation performance, regardless of the model used.

4.10 Statistical Comparison

Although ChatGPT achieved a slightly higher accuracy than Gemini, it is necessary to determine whether this observed difference is statistically significant or simply due to random variation. Since both models were evaluated on the same set of problems and each observation is classified as either correct or incorrect, the comparison was performed using McNemar's test.

The test is based on a contingency table that records, for each problem, whether both models produced correct solutions, both failed, or only one of the models generated a correct solution. This paired analysis enables the assessment of whether one model consistently outperforms the other across the evaluated problem set.

4.10.1 (i) Hypothesis

Null Hypothesis (H_0):

To be defined for the specific research question.

Alternative Hypothesis (H_1):

To be defined for the specific research question.

4.10.2 (ii) Statistical Test Selection

Selected Test: *To be defined*

The selected statistical test is justified based on the type of data (categorical or continuous), the experimental design (paired or independent samples), and the distributional properties of the dataset.

Data Assumption Checks:

- Normality assessment: *To be defined (e.g., Shapiro-Wilk test or visual inspection)*
- Distribution shape (skewness): *To be defined*

4.10.3 (iii) Data Characteristics

Normality Analysis:

To be defined after data collection

Skewness Analysis:

To be defined after data collection

4.10.4 (iv) Test Computation

The test statistic is computed using the following formulation:

Test statistic formula to be inserted here

Where:

- *Define variables used in the formula after data collection*

4.10.5 (v) Results

Test Statistic: *To be computed*

p-value: *To be computed*

Significance Level (α): 0.05

4.10.6 (vi) Decision and Conclusion

- If $p \leq \alpha$: Reject H_0
- If $p > \alpha$: Fail to reject H_0

Conclusion: *To be written after results are obtained*